

SDA — Inventory System Logic Specification

Purpose of this document: Hand this to a developer so they can rebuild (or extend) the inventory engine cleanly. It describes *what the system does and why* — the data model, the states an item moves through, and the exact rules that govern stock — extracted from the existing `ezsmurf-sda` codebase. It deliberately stays at the logic level and avoids the anti-bot/scraping plumbing, which is incidental to inventory correctness.

One-line summary: SDA is a multi-marketplace listing + fulfillment engine. A single pool of "accounts" (the real stock) is listed simultaneously on several marketplaces; the inventory logic's whole job is to keep one account from being sold twice, keep listing quantities in sync with real availability, and consume stock atomically when an order arrives.

1. System at a glance

Concept	What it is
Account	One unit of real stock — a single sellable game account (login/password/email).
Game	A product/SKU. In practice a <i>game + region + keyword</i> combination. Stock is the number of available Accounts pointing at it.
Order	A sale pulled from (or entered for) a marketplace. Consumes exactly one Account.
Platform	A marketplace SDA lists on: <code>eldorado</code> , <code>funpay</code> , <code>g2g</code> , <code>pa</code> (PlayerAuctions), <code>igv</code> , <code>csk</code> (own WooCommerce site). The codebase also carries <code>z2u</code> , <code>zeusx</code> , <code>ezsm</code> .
Slug	A specific seller <i>store account</i> on a platform (one platform can have several stores). Reservations and per-store pricing key off the slug.
Worker	A running process tied to one platform+slug. Holds that store's live session/credentials and does the listing + order polling.
Offer / OfferItem	A platform's listing record (e.g. <code>EldoradoOfferItem</code>) with its own action queue.

Runtime shape (current implementation)

Processes are launched by PM2 (`ecosystem.config.js`):

- **One worker process per platform+slug** — e.g. `(eldorado_CSK)`, `(pa_store2)`. Each polls its marketplace for new orders and pushes/refreshes listings.
- `(backend.js)` — Express REST API behind the Vue admin panel (auth, accounts, games, orders, stats).
- `(db-tasks.js)` — scheduled maintenance: stock balancing, cleanup, notifications, backups.
- `(telegram-bot.js)` — operator alerts/commands.

Stack: Node.js + Express, MongoDB via Mongoose (`(strict: false)` on Game and the OfferItem models, so platform-specific fields are added dynamically), Vue 2 admin frontend.

2. Core data model

This is the part to get right. Field names below match the existing schema so the developer can cross-reference.

2.1 Account (the stock unit)

Identity & secrets: `(login)`, `(password)`, `(email)`, `(emailPassword)`, `(securityQuestion)`, `(securityAnswer)`, `(emailLoginLink)`, `(screenshots[])`.

Classification:

- `(game)` → ref Game (which SKU this stock belongs to).
- `(user)` → ref User (who added it), `(supplier)` (free-text source).
- `(buyPrice)` (cost basis, used for profit math).

State flags (the important ones):

- `(flag): [0]` = in the available pool, `(1)` = consumed/attached to an order (out of pool).
- `(isFaulty)`: account was returned/replaced and is not sellable as-is.
- `(isSecured)`: account passed security setup (sellable).
- `(isPAFraud)`, `(isPAFaulty)`, `(isIGVFraud)`: platform-specific blacklist flags.

Reservation block (cross-platform locking):

- `(eldoradoReserved)`, `(paReserved)`, `(igvReserved)`, `(cskReserved)`: booleans.
- `(reservedBySlug)`: which store reserved it.
- `(dateReserved)`: when (used to age/expire reservations).

- `platformReference`, `secondaryPlatformReference`, `reservedIds[]`: the platform-side listing/offer IDs this account is tied to.

Bookkeeping: `failReason`, `failedOrderId`, `replacedBy[]`, `note`, `pushedToGoogleSheet`, `pushedToFaultyGoogleSheet`.

2.2 Game (the SKU)

- Identity: `title`, `name`, `keyword` (comma-separated matching terms — see §5.5), `region`, `url`, `color`, `dashboardOrder`.
- Pricing: `buyPrice`, `sellPrice`, plus **dynamic per-platform/per-store fields** (see §6).
- Stock policy: `lowStockQuantity`, `pauseLotsOnLowStock`, `bumpLots`.
- Listing surface: `soldOn[]` (which platforms this SKU is listed on).
- Lifecycle: `status` (`0` inactive, `1` active, `2` pending change), `pendingStatus`, `pendingStatusChangePerScript[]` (per-store pending toggles), `archived`, `lockedForEditing`.
- Misc: `eldoradoOffersCount`, `relatedOfferFoundOn[]`, `lastNotification`, `paListingPenaltyTill`, `lastInstantDeliveryRefresh`.

`strict: false` — the Game document is the central config blob; platforms read extra fields off it at runtime.

2.3 Order (the sale)

- Link: `game` → Game, `account` → Account (null until fulfilled), `user`.
- IDs: `orderId` (unique), `secondaryOrderId`, `thirdOrderId`, `offerId`, `lotId`.
- Routing: `platform`, `sellerSlug`, `keyword` (the keyword that matched), `buyer`.
- Money: `buyPrice`, `sellPrice`, `grossSellPrice`, `originalOrderQuantity`.
- `flag` (order lifecycle): `2` = new / no account attached yet (default), `0` = account attached, queued for delivery, `1` = delivered/complete.
- Type & handling: `type` (`sell`), `isManual`, `isReplaceable`, `hadReplacements`, `offerDeliveryType` (`instant`, `Minute5`, ...), `siteInstantDelivery`, `fiveMinDeliveryOrder`.
- Disputes: `isDisputed`, `isDisputeAutoReplied`.
- Flow tracking: `stateLogs[]`, `didMarkAsDelivered`, `pushedToGoogleSheet`, `failedToSendAccountCounter`.

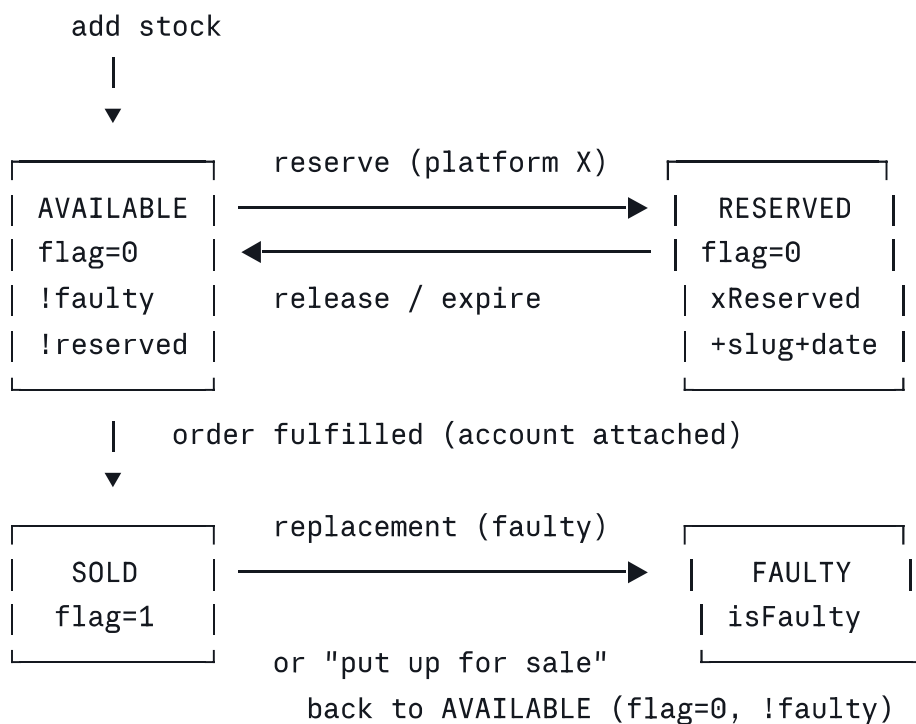
2.4 Supporting collections

- **GameSharedStockConnection** — `{ games: [GameId] }`. Groups SKUs that should share one stock pool (§5.8).

- **Offer** — temp record attaching an Account to a platform with reservation.
 - **EldoradoOfferItem** / **G2GOfferItem** / **IGVGameItem** / **PAPenalizedOfferItem** / **ZeusxRelistItem** / ... — per-platform listing rows. Key fields: `sdaId` (→ Game), `lastAction`, `nextAction[]`, `lastActionTs`, `lastError`, `quantity`, `priority`, `ignoreStatus`. These drive an action queue per listing.
 - **Worker** — per platform+slug runtime: session tokens/cookies, `isPaused`, `isSandboxed`, `lastIteration`.
 - **ActionHistory** — audit log per Account/Order (create, sale, replace, ...).
 - **User** — operators with permissions (`write_accounts`, `create_order`, `send_replacement`, `read_others_accounts`, ...) and `managedGames[]` scoping.
-

3. State machines

3.1 Account



- **AVAILABLE** is the only state counted as sellable stock (§5.2).
- **RESERVED** = still `flag=0` but soft-locked to one store; removed from the *shared* pool but usable by the reserving store.
- **SOLD** (`flag=1`) is terminal unless a replacement frees it.
- **FAULTY** is parked; an operator either fixes/replaces it or returns it to the pool.

3.2 Order

```
flag=2  (new, no account)          ← created on detection or manual entry
  |    account matched & attached
  ▼
flag=0  (account attached, queued for delivery)
  |    credentials delivered to buyer
  ▼
flag=1  (delivered / complete)
  |    buyer reports problem
  ▼
replacement: order → flag=2 (account detached), old account → FAULTY or pool,
              then new account attached → flag=0 → re-delivered → flag=1
```

`offerDeliveryType='instant'` orders can be picked up for delivery even at `flag=2`.

3.3 Game

`status`: `0` inactive → `2` pending (per-store toggles queued in `pendingStatusChangePerScript[]`) → `1` active. `archived=true` removes it from all listing/stock queries.

4. The canonical inventory invariants

A correct rebuild must preserve these. Everything in §5 exists to enforce them.

1. **One account = one sale.** An account in `flag=1` must never be deliverable again.
 2. **No cross-platform double-sell.** The same available account is listed on multiple marketplaces at once; only one marketplace may convert it. The reservation block is the lock.
 3. **Listed quantity ≈ real available quantity.** Each listing's pushed quantity is derived from the live available count, not stored independently.
 4. **Faulty/reserved stock is never counted as available.**
 5. **Stock-in and stock-out are auditable** (ActionHistory) and idempotent on `orderId` (unique) so a re-pollled marketplace order is never duplicated.
-

5. Inventory logic (the rules)

5.1 Stock-in (adding accounts)

Source: `backend/account/create.js`.

1. Operator pastes a blob of credentials, one account per line.
2. Each line is split by tab → space → colon, supporting 2-6 fields:
 - 2 = `login:password`
 - 3 = `login:password:emailPassword` (login doubles as email)
 - 4 = `login:password:email:emailPassword`
 - 5 = + `securityAnswer`
 - 6 = + `securityQuestion` + `securityAnswer`
3. Validate: no leading/trailing whitespace on login/password; reject in-batch duplicate logins; reject logins already present in DB.
4. Each account inherits: `game`, `buyPrice` (override or Game default), `supplier` (defaults to current user), `user`, optional `emailLoginLink`.
5. `insertMany`, write ActionHistory `create` rows, reset the Game's `lastNotification` and `paListingPenaltyTill`.
6. **Optional auto-activate:** if `shouldActivate` and the Game is inactive, set `status=2/pendingStatus=1` and queue per-store activation toggles for every platform in `soldOn` (excluding `pa`/`igv`/`csk`, which self-manage).

5.2 Available stock — the master formula

This single definition is used everywhere (`Account.getAvailableAccountsCount`, `get-stock.js`, every listing worker). **An account counts as available iff:**

```
game == <thisGame>
AND isFaulty == false
AND (flag == 0 OR flag missing)
AND eldoradoReserved is false/null/missing
AND paReserved is false/null/missing
AND igvReserved is false/null/missing
AND cskReserved is false/null/missing
```

The stock dashboard (`get-stock.js`) reports four numbers per Game via aggregation: `available`, `faulty`, `eldoradoReserved`, `paReserved`. **Implementation note for the rebuild:** the "reserved on any platform excludes from available" rule means a single `reservedBySlug` + a status enum would be cleaner than four parallel booleans (see §9).

5.3 Reservation (cross-platform double-sell prevention)

This is the heart of the system. Because the same pool is listed on many marketplaces, when a store wants to *physically commit* an account to one of its listings it **reserves** it:

```
set <platform>Reserved = true
set reservedBySlug      = <this store's slug>
set dateReserved        = now
(optional) platformReference = <listing id on that marketplace>
```

Effects:

- The account leaves the shared available pool (§5.2) → other platforms stop listing it → no double-sell.
- The reserving store keeps using it for its own listing/delivery.
- Reservations **age out**: workers re-scan accounts whose `dateReserved` is older than a threshold (e.g. PA extends/refreshes offers older than 21 days) and either refresh or release them (`reservedBySlug=''`, flag back to available).

Each platform has its own reserve setter (`utils/pa.js`, `utils/igv.js`, `utils/eldorado*.js`, `utils/ezsm.js`, ...) following the identical pattern with its own boolean.

5.4 Listing & quantity sync

Per platform worker, per Game it's responsible for:

1. Read `availableCount = getAvailableAccountsCount(gameId)`.
2. Read per-store policy off the Game: `accountCountToKeep_<platform>_<slug>` (reserve buffer), `offersCountToKeep_<platform>_<slug>` (how many live offers to maintain), `price_<platform>_<slug>`, plus title/description/server/gameName/messageTemplate (per-store override → per-platform default → blank).
3. Create/refresh listings up to the configured count; **push the listing quantity from the live available count**.
4. **Zero-stock guard**: if `availableCount == 0` and the offer isn't instant-delivery and `ignoreStatus` is false → do not list; record `lastError = "accounts left in stock is 0"` and stop. With `pauseLotsOnLowStock`, existing offers are paused when stock drops below `lowStockQuantity`.
5. Listing work is throttled (each worker iteration only processes a few games/offers) to stay under marketplace rate limits.

OfferItems carry an action queue (`lastAction`, `nextAction[]`) so operators/automation can enqueue actions (relist, reprice, pause) that the worker drains. Repricing supports

percentage syntax (e.g. `price = current * 1.05`).

5.5 Order intake & game matching

Source: `utils/database.js` → `pushOrderToDatabase`.

1. A worker polls its marketplace and gets a raw order (title, game name, quantity, price).
2. Idempotency: skip if `orderId` already exists (`hasOrderInDatabase`).
3. **Match the order to a Game by keyword:** load non-archived Games where `soldOn` includes this platform and `keyword` is set; for each Game, split `keyword` on commas and check whether the order title contains any keyword (case-insensitive, length > 1). First hit wins; the matched keyword is stored on the order.
4. **No match:** notify Telegram ("no game in database"), park the order against sentinel id `4edd40c86762e0fb12000003` with `flag=1`, and skip unless `ignore_uncofigured_orders` is set.
5. On match, seed `buyPrice` from the Game and `sellPrice` from `sellPrice_<platform>` if the marketplace didn't supply one.
6. **Quantity > 1 → split into N Order docs.** Copies after the first get a suffixed orderId (`<id>_-_1`, `_-_2`), `offerDeliveryType='Minute5'`, and no inherited account/buyPrice. `originalOrderQuantity` is preserved on each.
7. Sandbox mode (global `sandbox_mode` or per-worker `isSandboxed`) parks orders against the sentinel account so nothing real is consumed.

5.6 Fulfillment & stock consumption

Source: the delivery loop in each `utils/<platform>.js` (e.g. `utils/eldorado.js`).

1. Worker queries its deliverable orders: `sellerSlug == thisStore`, `type == 'sell'`, and (`flag=0` with account attached) OR (`flag=2` with `offerDeliveryType='instant'`) OR (auto-reply dispute cases).
2. For each, ensure an Account is chosen from the available pool for that Game, attach it to the order, and **atomically consume it:** `account.flag = 1`, order gets `account`/`account_login`, order `flag → 0` then on successful delivery (`→ 1`).
3. Deliver credentials to the buyer per the Game's `messageTemplate_<platform>`; optionally notify the buyer (`eldorado_notify_buyer_about_delivery`).
4. Mark order delivered (`didMarkAsDelivered`), push to Google Sheets if `google_sheets_push_orders`.
5. **Concurrency risk (flag for rebuild):** selection + `flag=1` is not a single atomic DB operation in the current code, so two near-simultaneous orders for the last unit can race. Use a `findOneAndUpdate` / conditional update / transaction to claim the account (§9).

5.7 Replacement & faulty handling

Source: `(backend/order/replace.js)`.

1. Operator supplies a fresh login/password/email + `(failReason)` (screenshot may be required by setting `(require_screenshot_for_replacement)`).
2. Non-admins can only replace orders newer than 7 days unless they have `(read_others_accounts)`.
3. The old account is either marked **FAULTY** (`(isFaulty=true)`) or, if "put up for sale" is checked, returned to the **pool** (`(isFaulty=false)`). Either way all reservation flags + `(reservedBySlug)` + `(platformReference)` are cleared, `(flag=0)`, and the supplied replacement credentials overwrite it; `(failedOrderId)` and `(replacedBy[])` are recorded.
4. The order's `(sellPrice)` is reduced by the account's `(buyPrice)` (cost of the replacement), `(hadReplacements++)`, `(flag=2)`, account detached — unless "put up for sale", which keeps the sale price.
5. If a specific `(replacementAccount)` id is given: re-point it to the order's Game if needed, attach it (order `(flag=0)`), and consume it (`(flag=1)`).
6. Telegram faulty alert if `(telegram_notify_about_faulty_accs)`.

5.8 Shared-stock balancing

Source: `(db-tasks.js → doStockBalancerStuff)`.

For each `(GameSharedStockConnection)` (a set of linked SKUs that should share inventory):

1. Compute each linked Game's available count.
2. Find the SKU with the highest count and the group average.
3. If average > 5, move excess accounts from the richest SKU to any SKU below average (transferring only when the move is meaningful, > 3 units), by reassigning `(account.game)`. This keeps several near-identical SKUs from going out of stock while another sits on a surplus.

5.9 Low-stock & lifecycle automation (db-tasks + workers)

- Low-stock detection drives Telegram alerts (`(telegram_notify_about_low_stock)`) and, with `(pauseLotsOnLowStock)`, pausing listings.
- Expired/aged offers get extended/refreshed (e.g. PA's 21-day window; Eldorado `(eldorado_extend_expired_offers)`, `(eldorado_relist_closed_offers)`).
- Cleanup jobs prune stale OfferItem/randomized-title rows by `(lastDataRefresh)`.
- Offline worker detection, exception alerts, periodic backups, sentinel-account handling for unconfigured orders.

6. Per-platform / per-store config model

Because Game is `{strict: false}`, listing config lives as dynamic keys on the Game document. The resolution order at runtime is **store-specific** → **platform-default** → **blank/fallback**:

```
game['price_pa_${slug}'] ?? game['price_pa'] ?? 5
game['title_pa_${slug}'] ?? game['title_pa'] ?? ''
```

Observed key families (suffix is the platform, optionally `[_<slug>]` for a specific store):

Key family	Meaning
<code>price_<platform>[_<slug>]</code>	listing price for that store
<code>title_<platform>[_<slug>]</code>	listing title
<code>description_<platform>[_<slug>]</code>	listing body
<code>server_<platform>[_<slug>]</code>	server/region attribute
<code>gameName_<platform>[_<slug>]</code>	platform's game name mapping
<code>messageTemplate_<platform>[_<slug>]</code>	delivery message to buyer
<code>accountCountToKeep_<platform>[_<slug>]</code>	stock buffer to hold back
<code>offersCountToKeep_<platform>[_<slug>]</code>	number of live offers to maintain
<code>sellPrice_<platform></code>	fallback sale price for order intake

For a clean rebuild, model these as a structured `{listingConfig: { [platform]: { [slug]: {...}, default: {...} } }}` rather than flat dynamic keys (§9).

7. Accounting / profit

- Per order: `buyPrice` (cost basis from the consumed account or Game) and `sellPrice` (net) and `grossSellPrice` (before fees). Profit = `sellPrice - buyPrice`; replacements subtract the replacement account's cost from `sellPrice`.
- Platform fees are applied (e.g. `eldorado_withdrawal_fee`).

- `backend/order/stats.js`, `profit.js`, `stats-export.js` aggregate over delivered orders (`flag=1`).
-

8. Settings & notifications

Settings are key/value (`app.settings.get(...)`). Inventory-relevant ones include: `sandbox_mode`, `ignore_uncofigured_orders`, `require_screenshot_for_replacement`, `google_sheets_push_accounts`, `google_sheets_push_orders`, `pauseLotsOnLowStock` (per Game), `eldorado_extend_expired_offers`, `eldorado_relist_closed_offers`, `eldorado_notify_buyer_about_delivery`, and the `telegram_notify_about_*` family (purchase, low_stock, faulty_accs, missing_game, missing_offers, offline_workers, deleted_accs, code_exceptions).

9. Recommendations for a clean rebuild

Things the current design does implicitly that a fresh inventory system should make explicit and safe:

1. **Atomic stock claim.** Replace "find available account, then set flag=1" with a single conditional update / transaction (`findOneAndUpdate({available}, {set sold})`) so concurrent orders for the last unit can't both win. This is the biggest correctness gap.
 2. **One reservation field, not four booleans.** Model account state as a single enum (`AVAILABLE | RESERVED | SOLD | FAULTY`) plus `reservedBy {platform, slug, ref, at}`. The "reserved on any platform" rule then becomes a single condition instead of a 4-way `$or` repeated across the codebase.
 3. **Structured listing config** (§6) instead of dynamic `key_platform_slug` strings on a `strict:false` document — easier to validate, diff, and migrate.
 4. **Derived quantity, never stored.** Keep treating listed quantity as a pure function of available count; never let a listing hold its own counter that can drift.
 5. **Idempotent intake** is already keyed on unique `orderId` — preserve that, and keep the quantity-split (one physical unit = one order row) so fulfillment stays 1:1 with stock.
 6. **Explicit reservation expiry** as a first-class scheduled job rather than per-platform ad-hoc age checks.
 7. **Audit everything** through one ActionHistory-style log (stock-in, reserve, release, sell, replace, faulty, transfer) — invaluable for disputes and reconciliation.
 8. **Sentinel account** (`4edd40c86762e0fb12000003`) for unmatched/sandbox orders is a useful pattern — keep an explicit "unallocated" bucket rather than nulls.
-

Quick glossary

- **flag (account):** 0 available · 1 sold.
- **flag (order):** 2 new/no account · 0 account attached, pending delivery · 1 delivered.
- **reserved:** soft-lock of an available account to one store to prevent cross-platform double-sell.
- **slug:** a specific seller store on a platform.
- **keyword:** comma-separated terms on a Game used to map an incoming order title to that SKU.
- **soldOn:** the platforms a Game is listed on.